

Tirocinio Interno — Laurea Triennale in Informatica

Sviluppo di JUnit in Python — Assignment 1

Fonte: JUnit User Guide — Versione 6.0.3

1. Obiettivo del progetto

Ogni funzionalità è stata testata praticamente con file di codice scritti e verificati localmente. Tutti i riferimenti alla guida ufficiale si basano sul documento: JUnit User Guide, Version 6.0.3.

2. Tabella di confronto — JUnit 6.0.3 vs Python (pytest)

Area	JUnit 6.0.3	Python / pytest	File di test
Annotazioni	@Test	def test_*	test_calcolatrice.py
Annotazioni	@DisplayName("...")	Docstring nella funzione	test_avanzati.py
Annotazioni	@Tag("nome")	@pytest.mark.nome	test_skip_mark.py
Annotazioni	@Nested	class TestNome: (classe annidata)	test_avanzati.py
Lifecycle	@BeforeEach	@pytest.fixture con yield (prima del yield)	test_fixture.py / test_lifecycle.py
Lifecycle	@AfterEach	@pytest.fixture con yield (dopo il yield)	test_lifecycle.py
Lifecycle	@BeforeAll	@pytest.fixture(scope="session")	test_lifecycle.py
Lifecycle	@AfterAll	fixture scope="session" + yield	test_lifecycle.py
Assertions	assertEquals(a, b)	assert a == b	test_calcolatrice.py
Assertions	assertTrue(x)	assert x	test_calcolatrice.py
Assertions	assertNull(x)	assert x is None	test_calcolatrice.py
Assertions	assertAll(...)	pytest-check: check.equal(...)	test_avanzati.py
Eccezioni	assertThrows(Tipo, codice)	with pytest.raises(Tipo):	test_calcolatrice.py
Eccezioni	assertThrowsExactly()	with pytest.raises(Tipo, match=...):	test_calcolatrice.py
Parametrizzazione	@ParameterizedTest + @ValueSource	@pytest.mark.parametrize("x", [1,2,3])	test_parametrize.py
Parametrizzazione	@CsvSource / @MethodSource	@pytest.mark.parametrize con lista di tuple	test_parametrize.py
Skip / Condizioni	@Disabled	@pytest.mark.skip(reason="...")	test_skip_mark.py
Skip / Condizioni	@EnabledOnOs / @DisabledOnOs	@pytest.mark.skipif(sys.platform==...)	test_skip_mark.py
Test suite	@Suite / @SelectClasses	pytest -m nome / selezione per path	test_skip_mark.py

3. Esempi pratici scritti durante lo studio

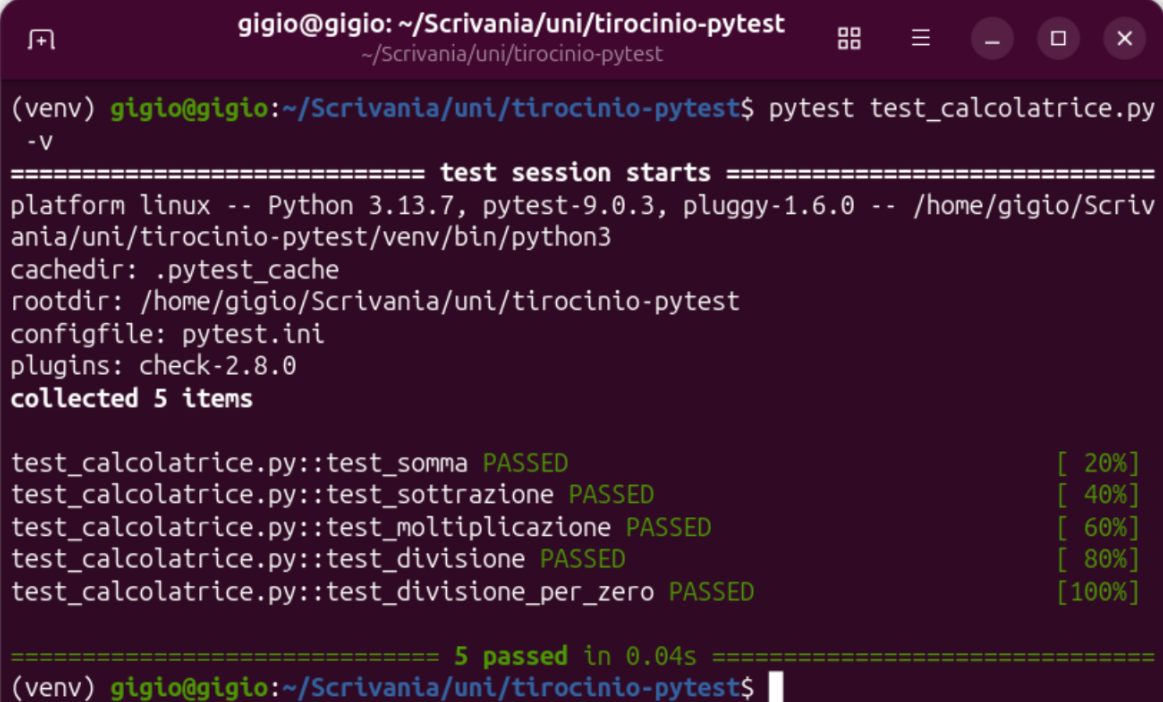
@Test + assertEquals + assertThrows

File: test_calcolatrice.py

```
import pytest
from calcolatrice import somma, divisione

def test_somma():
    assert somma(2, 3) == 5          # assertEquals(5, somma(2,3))

def test_divisione_per_zero():
    with pytest.raises(ValueError): # assertThrows(ValueError.class, ...)
        divisione(10, 0)
```

A terminal window with a dark purple background. The title bar shows the user 'gigio' at host 'gigio' in the directory '~/Scrivania/uni/tirocinio-pytest'. The terminal content shows the command 'pytest test_calcolatrice.py -v' being executed. The output includes a session start banner, environment details (Linux, Python 3.13.7, pytest 9.0.3), and a list of 5 tests: 'test_somma', 'test_sottrazione', 'test_moltiplicazione', 'test_divisione', and 'test_divisione_per_zero', all of which passed. A progress bar shows completion from 20% to 100%. The final summary states '5 passed in 0.04s'.

```
gigio@gigio: ~/Scrivania/uni/tirocinio-pytest
~/Scrivania/uni/tirocinio-pytest

(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$ pytest test_calcolatrice.py
-v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- /home/gigio/Scrivania/uni/tirocinio-pytest/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/gigio/Scrivania/uni/tirocinio-pytest
configfile: pytest.ini
plugins: check-2.8.0
collected 5 items

test_calcolatrice.py::test_somma PASSED [ 20%]
test_calcolatrice.py::test_sottrazione PASSED [ 40%]
test_calcolatrice.py::test_moltiplicazione PASSED [ 60%]
test_calcolatrice.py::test_divisione PASSED [ 80%]
test_calcolatrice.py::test_divisione_per_zero PASSED [100%]

===== 5 passed in 0.04s =====
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$
```

@BeforeEach / @AfterEach con yield

File: test_lifecycle.py

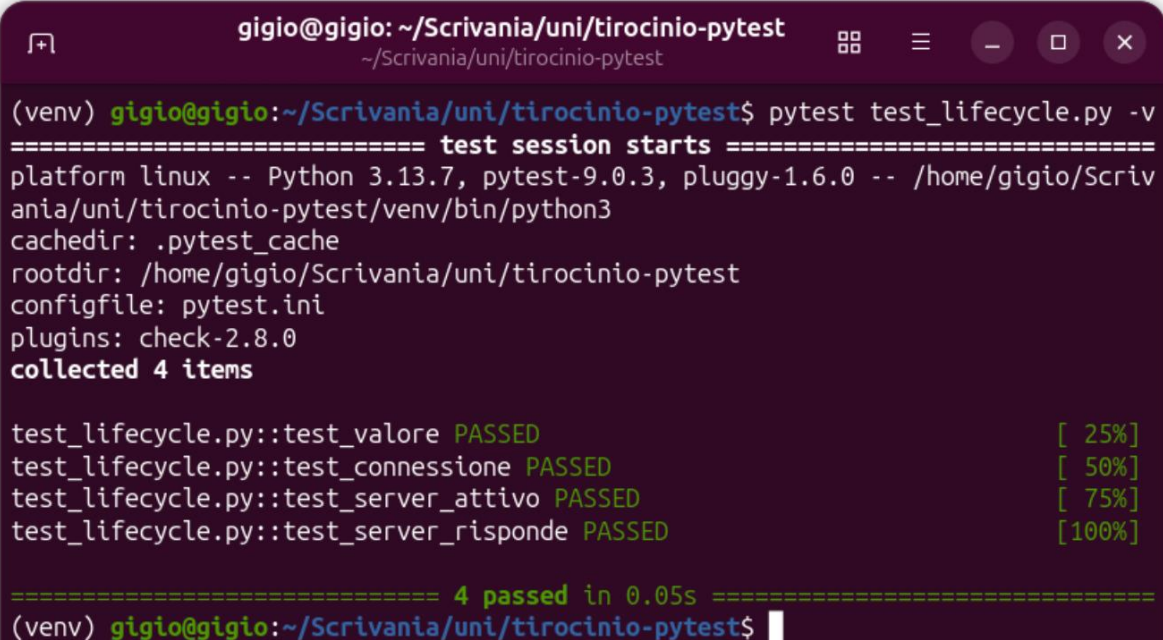
```
@pytest.fixture
def risorsa():
    dati = {"connesso": True}    # @BeforeEach
    yield dati                  # qui gira il test
    dati["connesso"] = False    # @AfterEach

def test_conessione(risorsa):
    assert risorsa["connesso"] == True
```

@BeforeAll / @AfterAll con scope="session"

File: test_lifecycle.py

```
@pytest.fixture(scope="session")
def server():
    stato = {"attivo": True}    # @BeforeAll - gira una sola volta
    yield stato
    stato["attivo"] = False    # @AfterAll - gira una sola volta
```



```
gigio@gigio: ~/Scrivania/uni/tirocinio-pytest
~/Scrivania/uni/tirocinio-pytest

(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$ pytest test_lifecycle.py -v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- /home/gigio/Scrivania/uni/tirocinio-pytest/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/gigio/Scrivania/uni/tirocinio-pytest
configfile: pytest.ini
plugins: check-2.8.0
collected 4 items

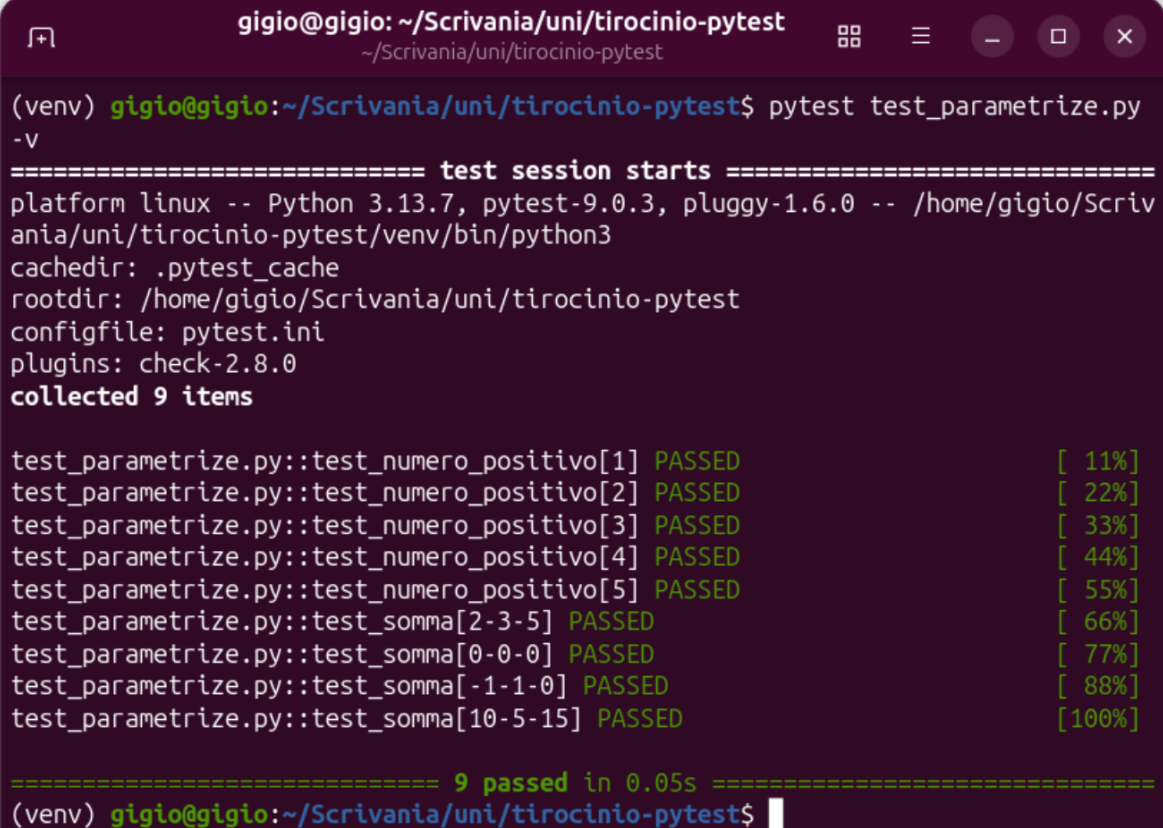
test_lifecycle.py::test_valore PASSED [ 25%]
test_lifecycle.py::test_conessione PASSED [ 50%]
test_lifecycle.py::test_server_attivo PASSED [ 75%]
test_lifecycle.py::test_server_risponde PASSED [100%]

===== 4 passed in 0.05s =====
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$
```

@ParameterizedTest + @CsvSource

File: test_parametrize.py

```
@pytest.mark.parametrize("a, b, atteso", [
    (2, 3, 5),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_somma(a, b, atteso):
    assert somma(a, b) == atteso
```



A terminal window titled "gigio@gigio: ~/Scrivania/uni/tirocinio-pytest" showing the execution of a pytest command. The command is "pytest test_parametrize.py -v". The output shows the test session starting with platform, Python version, pytest version, and other details. It then lists 9 collected items, all of which passed. The results are displayed in a table-like format with the test name, status, and percentage of completion. The final summary shows "9 passed in 0.05s".

```
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$ pytest test_parametrize.py
-v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- /home/gigio/Scrivania/uni/tirocinio-pytest/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/gigio/Scrivania/uni/tirocinio-pytest
configfile: pytest.ini
plugins: check-2.8.0
collected 9 items

test_parametrize.py::test_numero_positivo[1] PASSED [ 11%]
test_parametrize.py::test_numero_positivo[2] PASSED [ 22%]
test_parametrize.py::test_numero_positivo[3] PASSED [ 33%]
test_parametrize.py::test_numero_positivo[4] PASSED [ 44%]
test_parametrize.py::test_numero_positivo[5] PASSED [ 55%]
test_parametrize.py::test_somma[2-3-5] PASSED [ 66%]
test_parametrize.py::test_somma[0-0-0] PASSED [ 77%]
test_parametrize.py::test_somma[-1-1-0] PASSED [ 88%]
test_parametrize.py::test_somma[10-5-15] PASSED [100%]

===== 9 passed in 0.05s =====
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$
```

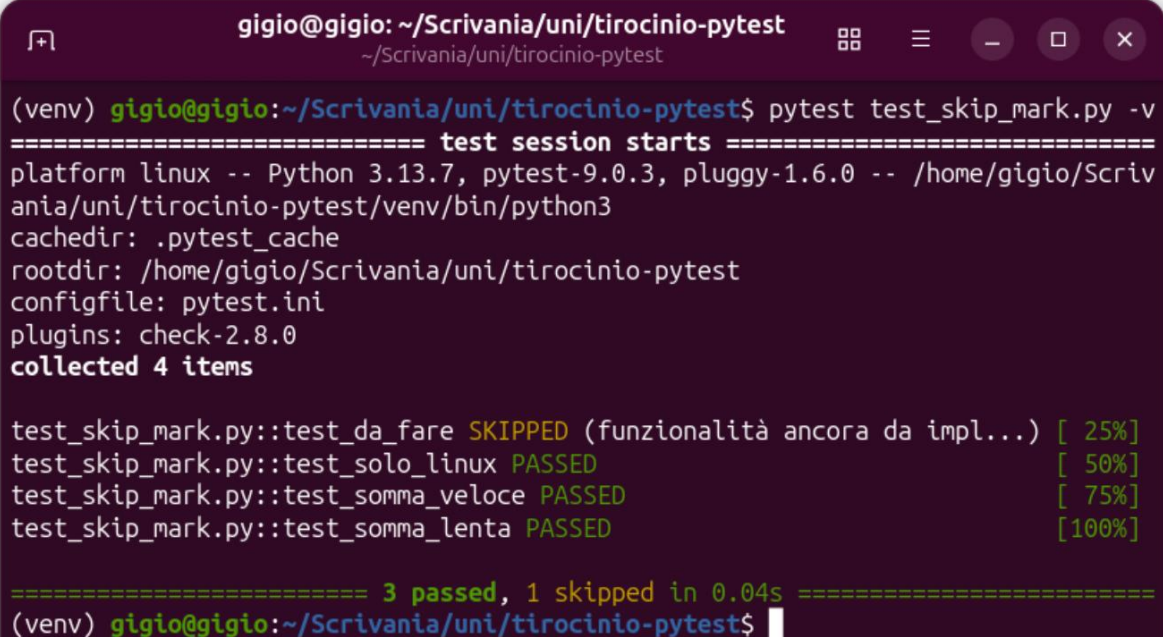
@Disabled + @Tag + @EnabledOnOs

File: test_skip_mark.py

```
@pytest.mark.skip(reason="da implementare") # @Disabled
def test_da_fare(): ...

@pytest.mark.veloce # @Tag("veloce")
def test_somma_veloce():
    assert somma(2, 3) == 5

@pytest.mark.skipif(sys.platform == "win32", # @DisabledOnOs
                    reason="solo Linux")
def test_solo_linux():
    assert somma(1, 1) == 2
```

A terminal window with a dark purple background. The title bar shows 'gigio@gigio: ~/Scrivania/uni/tirocinio-pytest'. The command 'pytest test_skip_mark.py -v' has been executed. The output shows session details, 4 items collected, and test results: 1 skipped (25%), 1 passed (50%), 1 passed (75%), and 1 passed (100%). The final summary shows 3 passed and 1 skipped in 0.04s.

```
gigio@gigio: ~/Scrivania/uni/tirocinio-pytest
~/Scrivania/uni/tirocinio-pytest

(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$ pytest test_skip_mark.py -v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- /home/gigio/Scriv
ania/uni/tirocinio-pytest/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/gigio/Scrivania/uni/tirocinio-pytest
configfile: pytest.ini
plugins: check-2.8.0
collected 4 items

test_skip_mark.py::test_da_fare SKIPPED (funzionalità ancora da impl...) [ 25%]
test_skip_mark.py::test_solo_linux PASSED [ 50%]
test_skip_mark.py::test_somma_veloce PASSED [ 75%]
test_skip_mark.py::test_somma_lenta PASSED [100%]

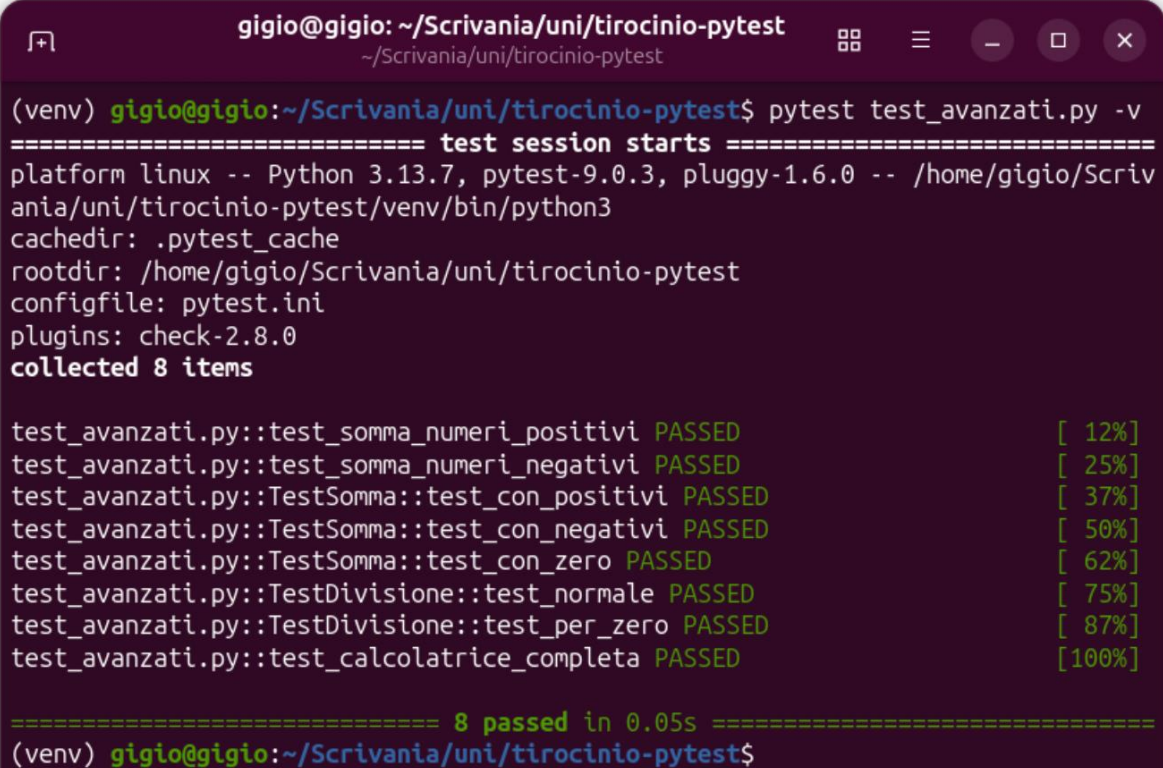
===== 3 passed, 1 skipped in 0.04s =====
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$
```

@Nested + @DisplayName + assertAll

File: test_avanzati.py

```
class TestSomma:
    """Gruppo test somma"""
    def test_positivi(self):
        assert somma(2, 3) == 5

def test_tutto_insieme():
    check.equal(somma(2, 3), 5)
    check.equal(divisione(10, 2), 5)
```



A terminal window titled "gigio@gigio: ~/Scrivania/uni/tirocinio-pytest" showing the execution of a pytest command. The output displays test session details, collected items, and a list of 8 passed tests with their progress percentages. The tests include individual function tests and a nested class test.

```
gigio@gigio: ~/Scrivania/uni/tirocinio-pytest
~/Scrivania/uni/tirocinio-pytest

(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$ pytest test_avanzati.py -v
===== test session starts =====
platform linux -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- /home/gigio/Scrivania/uni/tirocinio-pytest/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/gigio/Scrivania/uni/tirocinio-pytest
configfile: pytest.ini
plugins: check-2.8.0
collected 8 items

test_avanzati.py::test_somma_numeri_positivi PASSED [ 12%]
test_avanzati.py::test_somma_numeri_negativi PASSED [ 25%]
test_avanzati.py::TestSomma::test_con_positivi PASSED [ 37%]
test_avanzati.py::TestSomma::test_con_negativi PASSED [ 50%]
test_avanzati.py::TestSomma::test_con_zero PASSED [ 62%]
test_avanzati.py::TestDivisione::test_normale PASSED [ 75%]
test_avanzati.py::TestDivisione::test_per_zero PASSED [ 87%]
test_avanzati.py::test_calcolatrice_completa PASSED [100%]

===== 8 passed in 0.05s =====
(venv) gigio@gigio:~/Scrivania/uni/tirocinio-pytest$
```